```
"""Telegram-бот: скачивание видео, конвертер, QR, стикеры, погода. Владелец +
друзья по списку."""
import asyncio
import io
import json
import logging
import math
import os
import re
import shutil
import tempfile
import time as time_module
from collections import defaultdict
from datetime import time
from pathlib import Path
from urllib.parse import urlparse
from zoneinfo import ZoneInfo

import feedparser
import httpx
import qrcode
import yt_dlp
from dotenv import load_dotenv
from PIL import Image, ImageOps
from telegram import InlineKeyboardButton, InlineKeyboardMarkup,
ReplyKeyboardMarkup, Update
from telegram.ext import (
    Application, ApplicationHandlerStop, CallbackQueryHandler, CommandHandler,
    ContextTypes, MessageHandler, TypeHandler, filters,
)

load_dotenv()
BOT_TOKEN = os.environ["BOT_TOKEN"]
OWNER_ID = int(os.environ["OWNER_ID"])          # владелец: без лимитов, все
команды
CITY = os.getenv("CITY", "Москва")
FRIEND_CITY = os.getenv("FRIEND_CITY", "Москва")    # город друзей по умолчанию
MORNING_TIME = os.getenv("MORNING_TIME", "08:00")
TZ = ZoneInfo(os.getenv("TZ", "Europe/Moscow"))
LOCAL_API_URL = os.getenv("LOCAL_API_URL")       # напр. http://localhost:8081
(снимает лимит до 2 ГБ)
MAX_UPLOAD = (1900 if LOCAL_API_URL else 49) * 1024 * 1024   # макс. размер
одного файла
```

```
def _ids(raw: str | None) -> set[int]:
    return {int(x) for x in re.split(r"[,\s;]+", raw or "") if x.lstrip("-
").isdigit()}


ENV_ALLOWED = _ids(os.getenv("ALLOWED_IDS"))                    # друзья
«навсегда» из настроек Railway
DATA_DIR = Path(os.getenv("DATA_DIR", "data"))                  # для сохранения
друзей между перезапусками
STORE = DATA_DIR / "users.json"


def _load_users() -> dict:
    try:
        d = json.loads(STORE.read_text(encoding="utf-8"))
    except Exception:
        d = {}
    return {k: {int(i): n for i, n in d.get(k, {}).items()} for k in ("approved",
"blocked", "prefs")}


USERS = _load_users()                                           # approved /
blocked: {id: имя}


def _save_users():
    try:
        DATA_DIR.mkdir(parents=True, exist_ok=True)
        tmp = STORE.with_suffix(".tmp")
        tmp.write_text(json.dumps({k: {str(i): n for i, n in v.items()} for k, v
in USERS.items()},
                                  ensure_ascii=False), encoding="utf-8")
        tmp.replace(STORE)
    except Exception:
        log.exception("не удалось сохранить список друзей")


def city_of(uid: int) -> str:
    return CITY if uid == OWNER_ID else FRIEND_CITY


def pref(uid: int, key: str) -> bool:
    """Личные настройки: оповещения о дожде и утренняя сводка (по умолчанию
включены)."""
    return USERS["prefs"].get(uid, {}).get(key, True)
```

```
def subscribers(key: str) -> list[int]:
    ids = {OWNER_ID} | ENV_ALLOWED | set(USERS["approved"])
    return sorted(u for u in ids if pref(u, key))


def is_allowed(uid: int) -> bool:
    return uid == OWNER_ID or uid in ENV_ALLOWED or uid in USERS["approved"]


FRIEND_MAX_MIN = int(os.getenv("FRIEND_MAX_MIN", "30"))        # макс. длина
видео для друзей, мин
FRIEND_MAX_MB = int(os.getenv("FRIEND_MAX_MB", "150"))         # макс. размер
файла для друзей, МБ
FRIEND_JOBS_PER_HOUR = int(os.getenv("FRIEND_JOBS_PER_HOUR", "6"))   #
скачиваний/конвертаций в час
MAX_PARALLEL = int(os.getenv("MAX_PARALLEL", "2"))             # тяжёлых задач
одновременно на весь бот
FRIEND_DOMAINS = ("youtube.com", "youtu.be", "tiktok.com", "instagram.com")
Image.MAX_IMAGE_PIXELS = 40_000_000                            # защита от «бомб»
в картинках

NEWS_FEEDS = [                                     # можно заменить на любые RSS
    "https://feeds.bbci.co.uk/russian/rss.xml",
    "https://rss.dw.com/xml/rss-ru-all",
]
WEATHER_CODES = {
    0: "ясно ☀️", 1: "малооблачно 🌤️", 2: "облачно ⛅", 3: "пасмурно ☁️",
    45: "туман 🌫️", 48: "туман 🌫️", 51: "морось 🌦️", 53: "морось 🌦️", 55:
"морось 🌦️",
    61: "дождь 🌧️", 63: "дождь 🌧️", 65: "сильный дождь 🌧️", 71: "снег 🌨️",
    73: "снег 🌨️", 75: "сильный снег ❄️", 80: "ливень 🌧️", 81: "ливень 🌧️",
    82: "сильный ливень 🌧️", 95: "гроза ⛈️", 96: "гроза с градом ⛈️", 99: "гроза
с градом ⛈️",
}
URL_RE = re.compile(r"https?://\S+")
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %
(message)s")
log = logging.getLogger("bot")


# ---------- защита от перегрузки ----------
HEAVY = asyncio.Semaphore(MAX_PARALLEL)            # общая очередь тяжёлых задач
ACTIVE: set[int] = set()                           # у кого сейчас идёт задача
JOBS: dict[int, list[float]] = defaultdict(list)    # время запусков задач по
пользователям
MSGS: dict[int, list[float]] = defaultdict(list)    # время сообщений по
```

```
пользователям
STRANGERS: set[int] = set()                         # посторонние, которым уже
ответили


def is_owner(uid: int) -> bool:
    return uid == OWNER_ID


def url_allowed(url: str, uid: int) -> bool:
    """Друзьям можно только YouTube, TikTok и Instagram (чтобы бот не ходил на
чужие адреса)."""
    if is_owner(uid):
        return True
    host = (urlparse(url).hostname or "").lower()
    return any(host == d or host.endswith("." + d) for d in FRIEND_DOMAINS)


PENDING: dict[int, float] = {}                      # заявки, ждущие решения: id ->
время
DENIED: dict[int, float] = {}                       # отклонённые: id -> время
(повтор через сутки)
NAMES: dict[int, str] = {}                          # имена из заявок
NOTICED: dict[int, float] = {}                      # когда последний раз отвечали
«заявка уже отправлена»


async def handle_stranger(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Новый человек написал боту: отправляем владельцу заявку с кнопками."""
    user, msg, now = update.effective_user, update.effective_message,
time_module.time()
    uid = user.id
    for table, ttl in ((PENDING, 7 * 86400), (DENIED, 86400)):      # чистим
старое
        for k in [k for k, t in table.items() if now - t > ttl]:
            table.pop(k, None)
    if uid in PENDING or uid in DENIED:
        if msg and now - NOTICED.get(uid, 0) > 600:
            NOTICED[uid] = now
            await msg.reply_text("Заявка уже отправлена, дождитесь ответа
владельца." if uid in PENDING
                                 else "Владелец отклонил заявку. Повторить можно
через сутки.")
        return
    if len(PENDING) >= 20 or msg is None:
        return
    name = f"{user.full_name} (@{user.username})" if user.username else
```

```
user.full_name
    NAMES[uid] = name
    kb = InlineKeyboardMarkup([
        [InlineKeyboardButton("✅ Принять", callback_data=f"acc:ok:{uid}"),
         InlineKeyboardButton("❌ Отклонить", callback_data=f"acc:no:{uid}")],
        [InlineKeyboardButton("🚫 Заблокировать", callback_data=f"acc:ban:
{uid}")],
    ])
    try:
        await context.bot.send_message(OWNER_ID, f"🔔 Заявка на доступ к
боту\nИмя: {name}\nID: {uid}",
                                       reply_markup=kb)
    except Exception:
        log.exception("заявку не удалось отправить владельцу")
        await msg.reply_text("Не получилось отправить заявку, попробуйте позже.")
        return
    PENDING[uid] = now
    await msg.reply_text("Заявка отправлена владельцу бота. Я напишу, когда он
ответит.")


async def guard(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Срабатывает первым на любое обновление: пропускает только своих и режет
флуд."""
    user = update.effective_user
    if user is None:
        raise ApplicationHandlerStop
    if not is_allowed(user.id):
        if user.id not in USERS["blocked"] and not update.callback_query:
            await handle_stranger(update, context)
        raise ApplicationHandlerStop
    if not is_owner(user.id):
        now = time_module.time()
        recent = [t for t in MSGS[user.id] if now - t < 60]
        if len(recent) >= 20:                    # не больше 20 сообщений в
минуту
            MSGS[user.id] = recent
            raise ApplicationHandlerStop
        recent.append(now)
        MSGS[user.id] = recent


async def start_job(update: Update) -> bool:
    """Друзьям: одна задача за раз и не больше FRIEND_JOBS_PER_HOUR в час."""
    uid = update.effective_user.id
    if is_owner(uid):
        return True
```

```
    msg = update.effective_message
    if uid in ACTIVE:
        await msg.reply_text("Сначала дождитесь окончания предыдущей задачи.")
        return False
    now = time_module.time()
    recent = [t for t in JOBS[uid] if now - t < 3600]
    if len(recent) >= FRIEND_JOBS_PER_HOUR:
        wait = int((recent[0] + 3600 - now) // 60) + 1
        JOBS[uid] = recent
        await msg.reply_text(f"Лимит: {FRIEND_JOBS_PER_HOUR} задач в час.
Попробуйте через {wait} мин.")
        return False
    recent.append(now)
    JOBS[uid] = recent
    ACTIVE.add(uid)
    return True


def end_job(update: Update):
    ACTIVE.discard(update.effective_user.id)


def _limit_filter(info, *, incomplete=False):
    """Для друзей: не берём прямые эфиры и слишком длинные видео."""
    if info.get("is_live"):
        return "прямой эфир"
    duration = info.get("duration")
    if duration and duration > FRIEND_MAX_MIN * 60:
        return "слишком длинное видео"
    return None


# ---------- скачивание (yt-dlp) ----------
def _download(url: str, audio: bool, outdir: str, limited: bool = False) -> str:
    opts = {
        "outtmpl": f"{outdir}/%(id)s.%(ext)s",
        "noplaylist": True,
        "quiet": True,
        "socket_timeout": 30,
        "retries": 3,
    }
    if limited:
        opts["max_filesize"] = FRIEND_MAX_MB * 1024 * 1024
        opts["match_filter"] = _limit_filter
    if os.path.exists("cookies.txt"):            # нужен для части Instagram-
видео
        opts["cookiefile"] = "cookies.txt"
```

```
    if audio:
        opts["format"] = "bestaudio/best"
        opts["postprocessors"] = [{
            "key": "FFmpegExtractAudio", "preferredcodec": "mp3",
"preferredquality": "192",
        }]
    else:
        opts["format"] = "bv*[height<=720][ext=mp4]+ba[ext=m4a]/b[height<=720]/b"
        opts["merge_output_format"] = "mp4"
    with yt_dlp.YoutubeDL(opts) as ydl:
        info = ydl.extract_info(url, download=True)
        done = (info or {}).get("requested_downloads")
        if not done:
            raise RuntimeError(
                f"Видео не скачано: оно длиннее {FRIEND_MAX_MIN} мин, больше
{FRIEND_MAX_MB} МБ "
                "или это прямой эфир." if limited else "Видео не скачано.")
        return done[0]["filepath"]


async def split_file(path: str) -> list[str]:
    """Режет файл на части меньше MAX_UPLOAD без перекодирования."""
    proc = await asyncio.create_subprocess_exec(
        "ffprobe", "-v", "error", "-show_entries", "format=duration",
        "-of", "default=nw=1:nk=1", path,
        stdout=asyncio.subprocess.PIPE, stderr=asyncio.subprocess.DEVNULL,
    )
    out, _ = await proc.communicate()
    duration = float(out.decode().strip())
    parts = math.ceil(os.path.getsize(path) * 1.15 / MAX_UPLOAD)   # запас на
неровные куски
    p = Path(path)
    pattern = str(p.with_name(f"{p.stem}_part%03d{p.suffix}"))
    await ffmpeg("-i", path, "-c", "copy", "-f", "segment",
                 "-segment_time", str(int(duration / parts) + 1),
                 "-reset_timestamps", "1", pattern)
    return sorted(str(x) for x in p.parent.glob(f"{p.stem}_part*{p.suffix}"))


async def send_media(update: Update, path: str, audio: bool):
    files = [path]
    if os.path.getsize(path) > MAX_UPLOAD:
        await update.effective_message.reply_text("Файл большой, отправлю
частями.")
        files = await split_file(path)
    for i, part in enumerate(files, 1):
        caption = f"Часть {i}/{len(files)}" if len(files) > 1 else None
```

```
        with open(part, "rb") as f:
            if audio:
                await update.effective_message.reply_audio(f, caption=caption)
            else:
                await update.effective_message.reply_video(f, caption=caption,
supports_streaming=True)


async def download_and_send(update: Update, url: str, audio: bool = False):
    uid = update.effective_user.id
    if not url_allowed(url, uid):
        await update.effective_message.reply_text("Поддерживаются ссылки YouTube,
TikTok и Instagram.")
        return
    if not await start_job(update):
        return
    tmp = tempfile.mkdtemp()
    msg = None
    try:
        msg = await update.effective_message.reply_text(
            "⏳ В очереди, скоро начну..." if HEAVY.locked() else "⏳
Скачиваю...")
        async with HEAVY:
            path = await asyncio.to_thread(_download, url, audio, tmp, not
is_owner(uid))
            await send_media(update, path, audio)
        await msg.delete()
    except Exception as e:
        log.exception("download failed")
        if msg:
            await msg.edit_text(f"❌ Не получилось: {str(e)[:300]}")
    finally:
        shutil.rmtree(tmp, ignore_errors=True)
        end_job(update)


# ---------- конвертер (ffmpeg / Pillow) ----------
async def ffmpeg(*args: str, timeout: int = 1200):
    proc = await asyncio.create_subprocess_exec(
        "ffmpeg", "-y", *args,
        stdout=asyncio.subprocess.DEVNULL, stderr=asyncio.subprocess.PIPE,
    )
    try:
        _, err = await asyncio.wait_for(proc.communicate(), timeout)
    except asyncio.TimeoutError:
        proc.kill()
        await proc.wait()
```

```
        raise RuntimeError("Обработка заняла слишком много времени и была
остановлена.")
    if proc.returncode != 0:
        raise RuntimeError(err.decode()[-300:])


async def _convert_work(update: Update, tmp: Path, src: Path, mode: str):
    reply = update.effective_message
    if mode == "pdf":
        out = tmp / "result.pdf"
        Image.open(src).convert("RGB").save(out)
        await reply.reply_document(open(out, "rb"))
    elif mode == "sticker":
        img = ImageOps.exif_transpose(Image.open(src)).convert("RGBA")
        scale = 512 / max(img.size)                    # одна сторона ровно 512 px
        img = img.resize((max(1, round(img.width * scale)), max(1,
round(img.height * scale))),
                         Image.LANCZOS)
        out = tmp / "sticker.webp"
        for quality in (90, 75, 60, 40):               # стикер должен быть меньше
512 КБ
            img.save(out, "WEBP", quality=quality)
            if out.stat().st_size < 500 * 1024:
                break
        await reply.reply_sticker(open(out, "rb"))
    elif mode == "qrread":
        import cv2                                     # тяжёлая библиотека,
грузим только здесь
        text, _, _ = cv2.QRCodeDetector().detectAndDecode(cv2.imread(str(src)))
        await reply.reply_text(text or "QR-код на картинке не найден.")
    elif mode == "mp3":
        out = tmp / "result.mp3"
        await ffmpeg("-i", str(src), "-vn", "-acodec", "libmp3lame", "-q:a", "2",
str(out))
        await reply.reply_audio(open(out, "rb"))
    elif mode == "compress":
        out = tmp / "result.mp4"
        await ffmpeg("-i", str(src), "-vf", "scale=-2:720", "-vcodec", "libx264",
                     "-crf", "28", "-preset", "veryfast", "-acodec", "aac",
                     "-b:a", "96k", str(out))
        await reply.reply_video(open(out, "rb"))


async def convert(update: Update, context: ContextTypes.DEFAULT_TYPE, mode: str):
    fid = context.user_data.get("fid")
    if not fid:
        await update.effective_message.reply_text("Сначала пришли файл.")
```

```
        return
    if not await start_job(update):
        return
    tmp = Path(tempfile.mkdtemp())
    msg = None
    try:
        msg = await update.effective_message.reply_text(
            "⏳ В очереди, скоро начну..." if HEAVY.locked() else "⏳
Обрабатываю...")
        tg_file = await context.bot.get_file(fid)    # лимит скачивания у бота —
20 МБ
        if not is_owner(update.effective_user.id) and (tg_file.file_size or 0) >
FRIEND_MAX_MB * 1024 * 1024:
            raise RuntimeError(f"Файл больше {FRIEND_MAX_MB} МБ.")
        src = tmp / "input"
        async with HEAVY:
            await tg_file.download_to_drive(src)
            await _convert_work(update, tmp, src, mode)
        await msg.delete()
    except Exception as e:
        log.exception("convert failed")
        if msg:
            await msg.edit_text(f"❌ Ошибка: {str(e)[:300]}")
    finally:
        shutil.rmtree(tmp, ignore_errors=True)
        end_job(update)


# ---------- утренняя сводка ----------
async def _geocode(c: httpx.AsyncClient, city: str | None = None) -> dict:
    geo = (await c.get("https://geocoding-api.open-meteo.com/v1/search",
                       params={"name": city or CITY, "count": 1, "language":
"ru"})).json()
    if not geo.get("results"):
        raise LookupError(city or CITY)
    return geo["results"][0]


async def get_forecast(hours=(2, 4, 6), city: str | None = None) -> str:
    """Погода сейчас и через 2, 4, 6 часов."""
    async with httpx.AsyncClient(timeout=15) as c:
        place = await _geocode(c, city)
        w = (await c.get("https://api.open-meteo.com/v1/forecast", params={
            "latitude": place["latitude"], "longitude": place["longitude"],
            "current": "temperature_2m,weather_code",
            "hourly": "temperature_2m,precipitation_probability,weather_code",
            "timezone": "auto", "forecast_days": 2,
```

```
        })).json()
    cur, h = w["current"], w["hourly"]
    i = h["time"].index(cur["time"][:13] + ":00")          # индекс текущего часа
    lines = [f"🌍 {place['name']}",
             f"Сейчас: {cur['temperature_2m']:.0f} °C,
{WEATHER_CODES.get(cur['weather_code'], '')}"]
    for d in hours:
        j = i + d
        rain = h["precipitation_probability"][j] or 0
        lines.append(f"Через {d} ч ({h['time'][j][11:16]}): {h['temperature_2m']
[j]:.0f} °C, "
                     f"{WEATHER_CODES.get(h['weather_code'][j], '')}, осадки
{rain}%")
    return "\n".join(lines)


async def get_weather(city: str | None = None) -> str:
    async with httpx.AsyncClient(timeout=15) as c:
        place = await _geocode(c, city)
        w = (await c.get("https://api.open-meteo.com/v1/forecast", params={
            "latitude": place["latitude"], "longitude": place["longitude"],
            "daily":
"temperature_2m_max,temperature_2m_min,precipitation_probability_max,weathercode"
,
            "timezone": "auto", "forecast_days": 1,
        })).json()["daily"]
    desc = WEATHER_CODES.get(w["weathercode"][0], "")
    return (f"🌍 {place['name']}: {desc}\n"
            f"🌡️ {w['temperature_2m_min'][0]:.0f}…{w['temperature_2m_max']
[0]:.0f} °C, "
            f"осадки: {w['precipitation_probability_max'][0]}%")


def _fetch_news(per_feed: int = 3) -> str:
    lines = []
    for url in NEWS_FEEDS:
        for e in feedparser.parse(url).entries[:per_feed]:
            lines.append(f"• <a href=\"{e.link}\">{e.title}</a>")
    return "\n".join(lines) or "Новости сейчас недоступны."


async def morning_text(city: str | None = None, news: str | None = None) -> str:
    try:
        weather = await get_weather(city)
    except Exception:
        weather = "Погоду получить не удалось."
    if news is None:
```

```
        news = await asyncio.to_thread(_fetch_news)
    return f"☀️ <b>Доброе утро!</b>\n\n{weather}\n\n📰 <b>Главное в мире:
</b>\n{news}"


def morning_kb(uid: int) -> InlineKeyboardMarkup:
    label = "🔕 Не присылать по утрам" if pref(uid, "morning") else "🔔 Присылать
по утрам"
    return InlineKeyboardMarkup([[InlineKeyboardButton(label,
callback_data="menu:morning")]])


async def send_morning(context: ContextTypes.DEFAULT_TYPE):
    """Каждое утро: владельцу и друзьям, у кого рассылка не выключена."""
    news = await asyncio.to_thread(_fetch_news)
    cache: dict[str, str] = {}
    for uid in subscribers("morning"):
        city = city_of(uid)
        if city not in cache:
            cache[city] = await morning_text(city, news)
        try:
            await context.bot.send_message(uid, cache[city], parse_mode="HTML",
                                           disable_web_page_preview=True,
reply_markup=morning_kb(uid))
        except Exception:
            log.exception("утренняя сводка не доставлена: %s", uid)
        await asyncio.sleep(0.1)


# ---------- оповещение о дожде ----------
RAIN_MM = 0.1                    # осадков за 15 минут, с которых считаем, что
дождь начался
RAIN_COOLDOWN = 90 * 60          # не чаще одного оповещения в полтора часа


RAIN_STATE: dict[int, dict] = defaultdict(dict)     # у каждого свои: уже
оповещён, время последнего


async def check_rain(context: ContextTypes.DEFAULT_TYPE):
    """Раз в 10 минут смотрит прогноз на 15-минутных интервалах. Город
запрашиваем один раз на всех."""
    groups: dict[str, list[int]] = defaultdict(list)
    for uid in subscribers("rain"):
        groups[city_of(uid)].append(uid)
    for city, uids in groups.items():
        try:
```

```
            async with httpx.AsyncClient(timeout=15) as c:
                place = await _geocode(c, city)
                w = (await c.get("https://api.open-meteo.com/v1/forecast",
params={
                    "latitude": place["latitude"], "longitude":
place["longitude"],
                    "current": "temperature_2m",
                    "minutely_15": "precipitation,snowfall",
                    "timezone": "auto", "forecast_days": 2,
                })).json()
            m = w["minutely_15"]
            now = w["current"]["time"]
            idx = max((i for i, t in enumerate(m["time"]) if t <= now),
default=0)
            precip = [p or 0 for p in m["precipitation"]]
            snow = [x or 0 for x in m["snowfall"]]
        except Exception:
            log.exception("rain check failed: %s", city)
            continue

        window = precip[idx: idx + 9]             # текущий момент + следующие 2
часа
        if window[0] >= RAIN_MM or all(p < RAIN_MM for p in window):
            for uid in uids:                      # дождь уже идёт или не
ожидается: ждём следующего
                RAIN_STATE[uid]["alerted"] = False
            continue
        soon = [k for k in (1, 2) if precip[idx + k] >= RAIN_MM]   # через 15 и
30 минут
        if not soon:
            continue
        what = "пойдёт снег ❄️" if any(snow[idx + k] > 0 for k in soon) else
"пойдёт дождь ☔"
        for uid in uids:
            st = RAIN_STATE[uid]
            if st.get("alerted") or time_module.time() - st.get("last", 0) <
RAIN_COOLDOWN:
                continue
            try:
                await context.bot.send_message(
                    uid, f"Примерно через {soon[0] * 15} мин {what}
({place['name']})")
                st["alerted"] = True
                st["last"] = time_module.time()
            except Exception:
                log.exception("оповещение о дожде не доставлено: %s", uid)
```

```
async def rain_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    uid = update.effective_user.id
    on = not pref(uid, "rain")
    USERS["prefs"].setdefault(uid, {})["rain"] = on
    _save_users()
    await update.effective_message.reply_text(
        f"☔ Оповещения о дожде включены (город: {city_of(uid)}). Напишу за
полчаса до осадков."
        if on else "Оповещения о дожде выключены.")


# ---------- QR-коды ----------
async def send_qr(update: Update, text: str):
    text = text.strip()
    if not text:
        await update.effective_message.reply_text("Пришлите текст или ссылку для
QR-кода.")
        return
    if len(text) > 1000:
        await update.effective_message.reply_text("Слишком длинный текст для QR-
кода (максимум 1000 символов).")
        return
    try:
        qr = qrcode.QRCode(box_size=12, border=4)
        qr.add_data(text)
        qr.make(fit=True)
        buf = io.BytesIO()
        qr.make_image(fill_color="black", back_color="white").save(buf, "PNG")
        buf.seek(0)
        await update.effective_message.reply_photo(buf)
    except Exception:
        log.exception("qr failed")
        await update.effective_message.reply_text("Не получилось сделать QR-код
из этого текста.")


async def qr_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    parts = (update.message.text or "").split(maxsplit=1)
    if len(parts) < 2 or not parts[1].strip():
        await m_qr(update, context)                  # без текста: спросим, что
закодировать
        return
    await send_qr(update, parts[1])


# ---------- хендлеры ----------
```

```
BTN_VIDEO, BTN_FILE, BTN_QR = "📥 Видео по ссылке", "🖼️ Файл или фото", "🔲 QR-код"
BTN_WEATHER, BTN_HELP = "🌦️ Погода", "ℹ️ Помощь и правила"
BTN_MORNING, BTN_RAIN, BTN_FRIENDS = "☀️ Сводка", "☔ Дождь: вкл/выкл", "👥 Друзья"


def main_menu(uid: int) -> ReplyKeyboardMarkup:
    rows = [[BTN_VIDEO, BTN_QR], [BTN_WEATHER, BTN_FILE], [BTN_MORNING, BTN_RAIN], [BTN_HELP]]
    if is_owner(uid):
        rows[-1].append(BTN_FRIENDS)
    return ReplyKeyboardMarkup(rows, resize_keyboard=True, is_persistent=True,
                               input_field_placeholder="Пришлите ссылку или файл")


def welcome_text() -> str:
    file_mb = FRIEND_MAX_MB if LOCAL_API_URL else min(FRIEND_MAX_MB, 20)
    return (
        "👋 Привет! Вот что я умею:\n"
        "• Ссылка TikTok / Reels: пришлю видео без водяного знака\n"
        "• Ссылка YouTube: видео или MP3 на выбор\n"
        "• Видео-файл: сжать или вытащить MP3\n"
        "• Картинка: PDF, стикер или чтение QR-кода\n"
        "• /qr текст или ссылка: сделать QR-код\n"
        f"• /weather Город: погода сейчас и через 2, 4 и 6 часов (по умолчанию {FRIEND_CITY})\n"
        f"• ☀️ Сводка: погода и главные новости, каждое утро в {MORNING_TIME} приходит сама\n"
        "• ☔ Дождь: сообщу за полчаса до дождя или снега\n\n"
        "Правила и лимиты:\n"
        f"• не больше {FRIEND_JOBS_PER_HOUR} скачиваний и конвертаций в час, по одной за раз\n"
        f"• видео не длиннее {FRIEND_MAX_MIN} мин и не больше {FRIEND_MAX_MB} МБ, прямые эфиры не поддерживаются\n"
        f"• файлы для конвертации не больше {file_mb} МБ\n"
        "• ссылки только YouTube, TikTok и Instagram\n"
        "• не больше 20 сообщений в минуту\n"
        "• при большой нагрузке задачи ждут в очереди\n"
        "• утреннюю сводку и дождь можно выключить кнопками\n"
        "• владелец может отозвать доступ в любой момент"
    )


HELP = (
    "Что я умею:\n"
```

```
    "• Ссылка TikTok / Reels — пришлю видео без водяного знака\n"
    "• Ссылка YouTube — видео или MP3 на выбор\n"
    "• Видео-файл — сжать или вытащить MP3\n"
    "• Картинка — PDF, стикер или чтение QR-кода\n"
    "• /qr текст или ссылка — сделать QR-код\n"
    "• /weather — погода сейчас и через 2, 4 и 6 часов\n"
    "• /rain — включить или выключить оповещения о дожде (включены по
умолчанию)\n"
    "• Друзья из списка получают такие же сводку и оповещения по городу
FRIEND_CITY\n"
    "• /friends — список друзей: удалить или разблокировать\n"
    "• /add ID — добавить друга по ID вручную\n"
    f"• Каждое утро в {MORNING_TIME} — погода и новости (/morning — прямо
сейчас)"
)


async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    uid = update.effective_user.id
    await update.message.reply_text(HELP if is_owner(uid) else welcome_text(),
reply_markup=main_menu(uid))


async def send_weather(update: Update, context: ContextTypes.DEFAULT_TYPE, city:
str, remember: bool = False):
    try:
        text = await get_forecast(city=city)
        if remember:
            context.user_data["city"] = city            # запомним для следующего
раза
    except LookupError:
        text = "Не нашёл такой город. Нажмите «Другой город» и попробуйте
написать иначе."
    except Exception:
        log.exception("forecast failed")
        text = "Не удалось получить прогноз, попробуйте позже."
    kb = InlineKeyboardMarkup([[InlineKeyboardButton("🏙 Другой город",
callback_data="menu:city")]])
    await update.effective_message.reply_text(text, reply_markup=kb)


async def weather_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    arg = " ".join(context.args or []).strip()[:60]
    uid = update.effective_user.id
    city = arg or (CITY if is_owner(uid) else context.user_data.get("city") or
FRIEND_CITY)
    await send_weather(update, context, city, remember=bool(arg))
```

```
async def morning_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    uid = update.effective_user.id
    await update.effective_message.reply_text(
        await morning_text(city_of(uid)), parse_mode="HTML",
        disable_web_page_preview=True, reply_markup=morning_kb(uid))


async def on_text(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = (update.message.text or "").strip()
    action = MENU.get(text)
    if action:                                          # нажата кнопка меню
        context.user_data.pop("await", None)
        await action(update, context)
        return
    mode = context.user_data.pop("await", None)       # бот ждал ответа после
кнопки
    if mode == "qr":
        await send_qr(update, text)
        return
    if mode == "city":
        await send_weather(update, context, text[:60], remember=True)
        return
    m = URL_RE.search(text)
    if not m:
        await update.message.reply_text("Пришлите ссылку или файл либо выберите
действие в меню внизу.")
        return
    url = m.group(0)
    if not url_allowed(url, update.effective_user.id):
        await update.message.reply_text("Поддерживаются ссылки YouTube, TikTok и
Instagram.")
        return
    if "youtube.com" in url or "youtu.be" in url:
        context.user_data["url"] = url
        kb = InlineKeyboardMarkup([[
            InlineKeyboardButton("🎬 Видео", callback_data="yt:video"),
            InlineKeyboardButton("🎵 MP3", callback_data="yt:audio"),
        ]])
        await update.message.reply_text("Что скачать?", reply_markup=kb)
    else:                                               # TikTok, Instagram и всё
остальное
        await download_and_send(update, url)


async def on_media(update: Update, context: ContextTypes.DEFAULT_TYPE):
```

```
    msg = update.message
    context.user_data.pop("await", None)
    if msg.video:
        context.user_data["fid"] = msg.video.file_id
        buttons = [("📉 Сжать", "conv:compress"), ("🎵 В MP3", "conv:mp3")]
    elif msg.photo:
        context.user_data["fid"] = msg.photo[-1].file_id
        buttons = [("📄 В PDF", "conv:pdf"), ("🎭 Стикер", "conv:sticker"), ("🔲
Читать QR", "conv:qrread")]
    elif msg.document and (msg.document.mime_type or "").startswith("image/"):
        context.user_data["fid"] = msg.document.file_id
        buttons = [("📄 В PDF", "conv:pdf"), ("🎭 Стикер", "conv:sticker"), ("🔲
Читать QR", "conv:qrread")]
    else:
        await msg.reply_text("Такой формат пока не поддерживаю.")
        return
    kb = InlineKeyboardMarkup([[InlineKeyboardButton(t, callback_data=d) for t, d
in buttons]])
    await msg.reply_text("Что сделать?", reply_markup=kb)


async def on_button(update: Update, context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    if not is_allowed(q.from_user.id):
        return
    kind, action = q.data.split(":")
    if kind == "yt":
        url = context.user_data.get("url")
        if url:
            await download_and_send(update, url, audio=(action == "audio"))
    else:
        await convert(update, context, action)


async def _approve(context: ContextTypes.DEFAULT_TYPE, uid: int, name: str):
    USERS["approved"][uid] = name
    USERS["blocked"].pop(uid, None)
    PENDING.pop(uid, None)
    DENIED.pop(uid, None)
    _save_users()
    try:
        await context.bot.send_message(uid, "✅ Владелец принял вашу заявку!\n\n"
+ welcome_text(),
                                       reply_markup=main_menu(uid))
    except Exception:
        log.exception("не удалось отправить приветствие")
```

```
async def on_access(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Кнопки владельца: принять, отклонить, заблокировать, удалить,
разблокировать."""
    q = update.callback_query
    await q.answer()
    if not is_owner(q.from_user.id):
        return
    _, action, raw = q.data.split(":")
    uid = int(raw)
    name = NAMES.get(uid) or USERS["approved"].get(uid) or
USERS["blocked"].get(uid) or f"ID {uid}"
    if action == "ok":
        if uid in USERS["approved"]:
            await q.edit_message_text(f"✅ Уже принят: {name} (ID {uid})")
            return
        await _approve(context, uid, name)
        await q.edit_message_text(f"✅ Принят: {name} (ID {uid}). Приветствие с
правилами отправлено.")
    elif action == "no":
        PENDING.pop(uid, None)
        DENIED[uid] = time_module.time()
        await q.edit_message_text(f"❌ Отклонён: {name} (ID {uid})")
        try:
            await context.bot.send_message(uid, "Владелец отклонил заявку.
Повторить можно через сутки.")
        except Exception:
            pass
    elif action == "ban":
        PENDING.pop(uid, None)
        USERS["approved"].pop(uid, None)
        USERS["blocked"][uid] = name
        _save_users()
        await q.edit_message_text(f"🚫 Заблокирован: {name} (ID {uid}). Бот будет
его игнорировать.")
    elif action == "del":
        USERS["approved"].pop(uid, None)
        _save_users()
        await q.edit_message_text(f"🗑️ Доступ убран: {name} (ID {uid})")
        try:
            await context.bot.send_message(uid, "Владелец закрыл вам доступ к
боту.")
        except Exception:
            pass
    elif action == "unban":
        USERS["blocked"].pop(uid, None)
```

```
        _save_users()
        await q.edit_message_text(f"🔓 Разблокирован: {name} (ID {uid}). Он может подать заявку снова.")


async def friends_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    rows, lines = [], []
    for uid, name in USERS["approved"].items():
        lines.append(f"• {name} (ID {uid})")
        rows.append([InlineKeyboardButton(f"🗑 Убрать: {name[:25]}", callback_data=f"acc:del:{uid}")])
    for uid in sorted(ENV_ALLOWED):
        lines.append(f"• ID {uid} (из настроек Railway, убрать можно только там)")
    blocked = []
    for uid, name in USERS["blocked"].items():
        blocked.append(f"• {name} (ID {uid})")
        rows.append([InlineKeyboardButton(f"🔓 Разблокировать: {name[:20]}", callback_data=f"acc:unban:{uid}")])
    text = "👥 Друзья:\n" + ("\n".join(lines) if lines else "пока никого")
    if blocked:
        text += "\n\n🚫 Заблокированные:\n" + "\n".join(blocked)
    text += f"\n\nЗаявок в ожидании: {len(PENDING)}"
    await update.message.reply_text(text, reply_markup=InlineKeyboardMarkup(rows) if rows else None)


async def add_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    arg = (context.args[0] if context.args else "").strip()
    if not arg.isdigit():
        await update.message.reply_text("Напишите так: /add 123456789")
        return
    uid = int(arg)
    await _approve(context, uid, NAMES.get(uid) or f"ID {uid}")
    await update.message.reply_text(f"✅ Добавлен ID {uid}. Если человек ещё не запускал бота, "
                                    "приветствие не дойдёт: пусть сначала нажмёт «Запустить».")


async def m_video(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Пришлите ссылку на видео из YouTube, TikTok или Instagram.")


async def m_file(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
```

```
        "Пришлите видео или картинку, и я предложу действия:\n"
        "• видео: сжать или вытащить MP3\n"
        "• картинка: PDF, стикер или чтение QR-кода")


async def m_qr(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data["await"] = "qr"
    await update.effective_message.reply_text(
        "Пришлите текст или ссылку, и я сделаю QR-код.\n"
        "Чтобы прочитать готовый QR, пришлите его фото и нажмите «Читать QR».")


async def m_help(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await start(update, context)


def owner_only(fn):
    async def wrapper(update: Update, context: ContextTypes.DEFAULT_TYPE):
        if is_owner(update.effective_user.id):
            await fn(update, context)
    return wrapper


MENU = {                                          # текст кнопки -> действие
    BTN_VIDEO: m_video, BTN_FILE: m_file, BTN_QR: m_qr,
    BTN_WEATHER: weather_cmd, BTN_HELP: m_help,
    BTN_MORNING: morning_cmd, BTN_RAIN: rain_cmd,
    BTN_FRIENDS: owner_only(friends_cmd),
}


async def on_menu(update: Update, context: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    uid = q.from_user.id
    if not is_allowed(uid):
        return
    if q.data == "menu:city":
        context.user_data["await"] = "city"
        await q.message.reply_text("Напишите название города, например: Казань")
    elif q.data == "menu:morning":
        on = not pref(uid, "morning")
        USERS["prefs"].setdefault(uid, {})["morning"] = on
        _save_users()
        try:
            await q.edit_message_reply_markup(reply_markup=morning_kb(uid))
        except Exception:
```

```
            pass
        await q.message.reply_text(f"Утренняя сводка в {MORNING_TIME} включена."
if on
                                   else "Утренняя сводка выключена. Кнопка «☀️
Сводка» работает как раньше.")


def main():
    builder = (Application.builder().token(BOT_TOKEN)
               .concurrent_updates(True)             # долгая загрузка не
блокирует остальные команды
               .read_timeout(1800).write_timeout(1800).connect_timeout(60))
    if LOCAL_API_URL:
        builder = (builder.base_url(f"{LOCAL_API_URL}/bot")
                   .base_file_url(f"{LOCAL_API_URL}/file/bot")
                   .local_mode(True))
    app = builder.build()
    app.add_handler(TypeHandler(Update, guard), group=-1)   # проверка доступа и
лимитов первой
    owner = filters.User(OWNER_ID)                           # только владелец:
/morning, /rain
    allowed = filters.ALL                                    # доступ уже
проверил guard
    app.add_handler(CommandHandler(["start", "help", "menu"], start, allowed))
    app.add_handler(CommandHandler("morning", morning_cmd, allowed))
    app.add_handler(CommandHandler("weather", weather_cmd, allowed))
    app.add_handler(CommandHandler("qr", qr_cmd, allowed))
    app.add_handler(CommandHandler("rain", rain_cmd, allowed))
    app.add_handler(CommandHandler("friends", friends_cmd, owner))
    app.add_handler(CommandHandler("add", add_cmd, owner))
    app.add_handler(MessageHandler(allowed & filters.TEXT & ~filters.COMMAND,
on_text))
    app.add_handler(MessageHandler(allowed & (filters.VIDEO | filters.PHOTO |
filters.Document.ALL), on_media))
    app.add_handler(CallbackQueryHandler(on_access, pattern=r"^acc:"))
    app.add_handler(CallbackQueryHandler(on_menu, pattern=r"^menu:"))
    app.add_handler(CallbackQueryHandler(on_button, pattern=r"^(yt|conv):"))
    hh, mm = map(int, MORNING_TIME.split(":"))
    app.job_queue.run_daily(send_morning, time(hh, mm, tzinfo=TZ))
    app.job_queue.run_repeating(check_rain, interval=600, first=30)   # проверка
дождя каждые 10 минут
    app.run_polling()


if __name__ == "__main__":
    main()
```